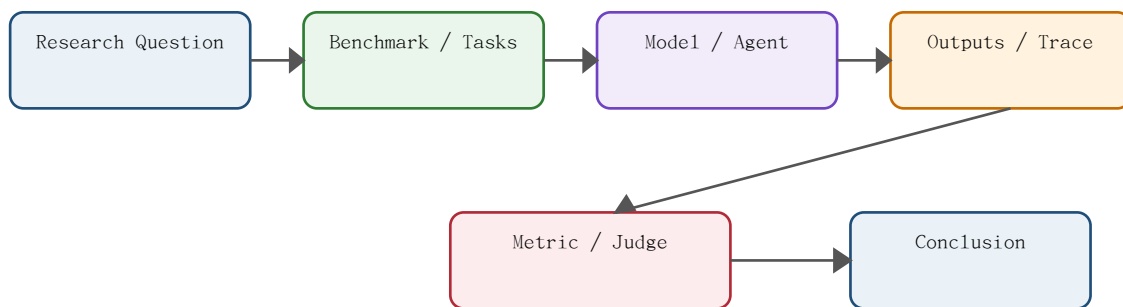


第31讲：LLM Evaluation

Benchmark、Pass@k、LLM-as-a-Judge 与评测陷阱

本讲目标

学完后你应该能回答：Benchmark 到底是什么；Accuracy、Exact Match、F1、Pass@k 各在测什么；什么时候需要 Human Evaluation；LLM-as-a-Judge 为什么方便但有偏差；Benchmark Contamination 怎样破坏结论；以及 Agent Systems 为什么必须同时评测结果、过程与成本。



评测不是“算一个分数”，而是把研究问题操作化为任务、输出、度量和结论。

先记住一句：评测不是“挑一个分数”，而是把研究问题转化成可重复、可比较、可审计的测量程序。

1. Benchmark 到底是什么？

Benchmark 可以理解成一套标准化测试环境：包含任务/样本、输入格式、参考答案或判定规则，以及最终指标。它的价值不是“绝对证明模型聪明”，而是让不同方法在相对一致的条件下比较。

组成	作用
Dataset / Tasks	规定测什么题
Prompt / Protocol	规定怎样提问、给哪些上下文
Reference / Judge	规定怎样判断输出
Metric	规定怎样把结果变成数字
Split	规定开发集、测试集如何隔离

重要

Benchmark

只是现实能力的一个投影。分数高说明在该测试协议下表现好，不等于“在所有真实场景都更强”。

2. Evaluation 的第一原则：先问“我到底想测什么？”

不同研究问题需要不同指标。比如：

研究问题	更直接的指标
事实问答是否正确？	Accuracy / Exact Match
分类是否平衡？	Precision / Recall / F1
代码能否至少生成一个可通过方案？	Pass@k
开放式回答哪个更好？	Pairwise Preference / Human / Judge
Agent 能否完成任务？	Task Success + Trajectory + Cost
多 Agent 是否产生集体收益？	Collective Gain + Error Correlation + Compute

科研习惯

指标应该由研究问题决定，而不是因为某指标容易算就拿来当主指标。

3. Accuracy：最直观，但不总够用

Accuracy = 正确样本数 / 总样本数。

Accuracy = correct / total

如果 100 道题答对 83 道，Accuracy = 83%。

它适合“每题有明确对错”的任务，但当类别严重不平衡、输出部分正确、或开放式答案存在多种合法表达时，Accuracy 就可能太粗。

4. Exact Match：为什么开放文本很容易被它“误伤”？

Exact Match (EM) 要求模型输出与标准答案经过规范化后完全匹配。

Reference: 25 Output A: 25 -> match Output B: The answer is 25. -> may not match

所以 EM 很严格，优点是客观、容易复现；缺点是对等价表达不够宽容。

场景
数学短答案、实体名、结构化字段适合 EM；长解释、摘要、开放问答通常不应只靠 EM。

5. Precision、Recall、F1：什么时候要看它们？

对二分类任务，假设“阳性”是你真正关心的事件：

Precision = TP / (TP + FP) Recall = TP / (TP + FN) F1 = 2 * Precision * Recall / (Precision + Recall)	
指标	直觉
Precision	我说“是”的里面，有多少真的“是”？
Recall	所有真正“是”的里面，我找回了多少？
F1	Precision 与 Recall 的折中

这和第26讲 RAG 的 Recall/Precision

思路完全一致：检索也可以被看作“找相关证据”的分类/排序问题。

6. Pass@k: 为什么代码模型常用它?

代码生成很特别：一次采样失败，不代表模型完全不会。只要给 k 次机会，其中至少一次通过测试，就算 Pass@ k 成功。

如果每次独立成功概率为 p ，一个简单理论直觉是：

$$\text{Pass@k} = 1 - (1 - p)^k$$

例如 $p=0.30$ ：

k	理论 Pass@k
1	0.300
2	0.510
5	0.832
10	0.972

关键

Pass@ k 把“模型能力”和“采样预算”绑在一起。 k 变大，test-time compute 也变大，所以比较时必须保持 k 或总预算一致。

7. Empirical Pass@k: 论文里为什么不是简单算 $1-(1-p)^k$?

真实评测通常先对每个题采样 n 个候选，其中 c 个通过测试，再估计如果随机挑 k 个，至少一个正确的概率。常见估计形式：

$$\text{pass@k} = 1 - C(n-c, k) / C(n, k)$$

如果 $n-c < k$ ，则 $\text{pass@k} = 1$ 。

不要死记公式

你现在只需要理解：它是在有限样本中估计“给 k 次尝试机会，至少有一个成功”的概率。第35讲会再讲重复实验与统计不确定性。

8. 开放式任务：为什么很难有唯一标准答案?

对于解释、摘要、建议、论文审阅、Agent 计划等任务，通常不存在唯一字符串答案。于是评测会转向：

方法	优点	缺点
规则/Checklist	可解释、可重复	覆盖有限
Reference-based score	自动化方便	参考答案可能不唯一
Human Evaluation	最接近真实偏好	贵、慢、有人际差异
LLM-as-a-Judge	便宜、可扩展	Judge 也有偏差
Task Outcome	直接看是否完成	不一定解释过程质量

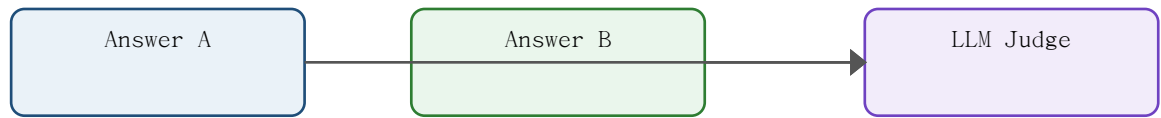
9. Pairwise Evaluation: A 和 B 哪个更好?

相比给每个答案打 1-10 分，Pairwise 常让 Judge 直接选择 A 或 B。

```
Prompt + Answer A + Answer B -> Judge -> A / B / Tie
```

这种比较通常比绝对打分更容易稳定，但仍有顺序偏差、长度偏差等问题。

10. LLM-as-a-Judge: 为什么这么常见？



Position Bias · Verbosity Bias · Style Bias · Self-preference

所以: Judge 也需要校准、交换顺序、多人/多模型交叉验证。

“有 Judge” 不等于 “有真值” 。

LLM Judge 可以自动评大量开放式输出，尤其适合需要语言理解但难写规则的任务。

但它不是“真值机器”，至少可能有：

偏差	现象
Position Bias	更偏向 A 或第一位置
Verbosity Bias	更长、更详细的回答被偏爱
Style Bias	更专业/更礼貌的语言被高估
Self-preference	Judge 偏好与自己风格/家族接近的输出
Reference Bias	参考答案本身有偏差时被放大

最低防护

交换 A/B 顺序、允许 Tie、多 Judge 交叉、用一部分人工标注校准 Judge，并保存 Judge prompt/version。

11. Human Evaluation: 人类就一定可靠吗?

也不一定。人类评审会受疲劳、专业背景、题目理解和个人偏好影响。

设计问题	更稳妥做法
标准模糊	给出明确 rubric 与示例
单人偏差	多评审
顺序偏差	随机化 A/B 顺序
任务太长	分解成原子维度
评审不一致	报告一致性/仲裁规则

因此 Human Evaluation 也需要协议设计，不能简单写“由人类判断更好”。

12. Benchmark

Contamination: 测试题如果被模型见过会怎样?

如果 benchmark

的题目、答案或高度相似变体出现在训练数据中，测试就可能测到“记忆过”而不是泛化。

污染形式	例子
Exact contamination	题目和答案原样进入训练集
Near-duplicate	轻微改写但结构相同
Solution leakage	公开解答被训练数据收录
Prompt tuning leakage	开发时反复针对测试集调 prompt

科研纪律

测试集应该尽量保持封闭。反复看测试集、针对测试失败不断改方法，本质上也会把测试集变成开发集。

13. Benchmark Saturation: 分数接近满分以后怎么办?

当大量强模型在 benchmark 上接近满分时，分数差异可能越来越难解释，且 benchmark 可能无法覆盖更难的新能力。

常见应对方式：更难任务、动态 benchmark、交互式环境、过程指标、对抗样本、成本约束。

注意

“旧 benchmark 饱和”不意味着模型已解决现实世界全部问题，只说明该测试的区分能力在下降。

14. Goodhart's Law: 指标一旦成为目标会发生什么?

一个常见表述是：当一个指标成为强优化目标，它就更可能失去作为真实目标代理的价值。

例如：

Optimize benchmark score -> exploit quirks / overfit format -> score rises, real utility may not

这也是为什么不能只盯一个 leaderboard 分数。

15. Agent Evaluation 为什么比普通 LLM 更难？

Agent 输出的不只是一个答案，而是一条多步 trajectory。最终结果可能正确，但过程极其低效或危险。

Agent 评测层	指标示例
Outcome	Task Success / Final Correctness
Process	Steps / Invalid Actions / Recovery
Tool	Tool Selection / Argument / Execution Success
State	State Accuracy / Progress Tracking
Cost	Tokens / Calls / Tool Calls / Latency / USD
Safety	Unauthorized Action / Side-effect Error

核心

对 Agent 论文来说，只报 Task Success 通常不够。至少还要看过程 and 成本。

16. Multi-Agent Evaluation: 最容易被“多花算力”骗

如果多 Agent 比单 Agent 更强，必须问：

问题	为什么重要
总 model calls 一样吗？	多调用本身会提高成功机会
总 output tokens 一样吗？	更长推理就是更多 test-time compute
Tool calls 一样吗？	更多外部试错也会提高成功率
Judge / verifier 一样吗？	额外 verifier 本身有收益
错误相关性怎样？	决定集体投票是否有效

所以第29讲的 compute-matched baseline 也是 Evaluation 设计的一部分。

17. 一张研究接口表：科研问题 -> 指标 -> 混杂

科研问题	主指标	必须控制的混杂
Memory 是否有帮助？	Task Success / Retrieval Hit	Context token、额外调用
通信是否有帮助？	Collective Gain / Info Gain	通信 token、轮数
异质性是否有帮助？	Error Correlation / Gain	模型强度、prompt、工具
Verifier 是否有帮助？	Error Detection / Final Accuracy	额外 compute
Planning 是否有帮助？	Success / Steps / Recovery	输出 token、更多重试
RAG 是否有帮助？	Recall@k / Faithfulness / Accuracy	文档库、chunk、context budget

18. 一个最低可接受的实验报告应该包含什么？

类别	至少报告
Model	模型名/版本/主要解码参数
Prompt	System prompt / template / tool schema
Data	任务、split、是否可能污染
Protocol	每题运行几次、k、轮数、最大 steps
Metrics	主指标 + 过程指标 + 成本指标
Judge	Judge 模型、prompt、顺序处理
Budget	tokens / calls / tools / latency
Failures	失败类型和至少部分案例分析

可复现性
以后你自己写 Agent Systems
论文，最好把这些内容形成固定实验模板。这样不容易漏掉审稿人最关心的控制变量。

19. 小实验: Pass@k 与 Judge Bias

材料包的 Python 实验包含两个部分。

实验 A: Pass@k – 固定单次成功概率, 观察 k 增大时“至少一次成功”的概率怎样上升。

实验 B: Judge Bias – 构造两个质量相同但长度不同的回答, 让一个带 verbosity bias 的 Toy Judge 比较, 并通过交换 A/B 顺序观察 position bias。

观察重点

Pass@k 上升并不表示单次能力变强; Judge 偏好长答案也不表示长答案更正确。评测协议本身会改变结论。

20. 常见误区汇总

误区	更准确的理解
Benchmark 高分 = 现实一定更强	只说明该协议下更强
Exact Match 低 = 回答一定差	可能只是等价表达不同
Pass@10 高 = 单次能力高	k 本身就是额外预算
LLM Judge = 自动真值	Judge 也有系统偏差
Human Judge = 无偏	人类也需要 rubric 和一致性控制
测试集反复调参没关系	会把测试集变成开发集
Agent 成功率高就够了	还要看过程、成本、安全

21. 练习题

- 1) Benchmark 的核心组成有哪些?
- 2) Accuracy 和 Exact Match 有什么区别?
- 3) Precision、Recall、F1 分别在测什么?
- 4) Pass@k 为什么不能脱离 k 单独比较?
- 5) LLM-as-a-Judge 最常见的偏差有哪些?
- 6) Human Evaluation 为什么也需要严格协议?
- 7) 什么叫 Benchmark Contamination?
- 8) Agent 评测为什么必须同时看 Outcome、Process 和 Cost?
- 9) 多 Agent 实验中哪些资源变量必须控制?
- 10) 如果你研究“通信能否降低错误相关性”, 你会用什么主指标和 baseline?

答案见材料包《03_第31讲_练习题与答案.pdf》。

22. 本讲总结

一句话总纲

好的 Evaluation 不是给系统贴一个分数，而是设计一套能回答研究问题、区分机制、控制混杂并可重复执行的测量程序。

对象	典型指标
闭卷事实/分类	Accuracy / EM / F1
代码/多采样	Pass@k
开放文本	Pairwise / Human / LLM Judge
RAG	Recall@k / Faithfulness / Citation
Agent	Success + Trajectory + Cost
Multi-Agent	Collective Gain + Error Correlation + Compute

下一讲：第32讲 《Uncertainty & Calibration: 模型到底知不知道自己不知道? 》。